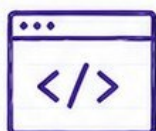


TERRAFORM

IN A NUTSHELL



From Infrastructure as Code Fundamentals
to Production Terraform



WRITE
CODE



PLAN
CHANGES



PROVISION
INFRASTRUCTURE



MANAGE
SAFELY



UPDATE &
ROLLBACK



WHAT YOU WILL MASTER

- ✓ Terraform Fundamentals | Providers & Resources | Variables & Outputs
- ✓ State Management | Modules | Loops & Conditionals | Dependencies
- ✓ Security & Best Practices | CI/CD Integration | Troubleshooting
- ✓ Production Architecture | Real-World Labs

CODE IT. PLAN IT. PROVISION IT.
MANAGE IT. OWN IT.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TERRAFORM EXPLAINED, WHY ENGINEERS USE IT

1 WHAT TERRAFORM IS

Terraform is an Infrastructure as Code (IaC) tool that helps you build, change, and version cloud and on-prem infrastructure safely and predictably.

You describe your infrastructure in code. Terraform figures out how to make it real.



Terraform
INFRASTRUCTURE AS CODE

2 INFRASTRUCTURE AS CODE (IaC)

- You write code (declarative) to describe the infrastructure.
- The code is versioned, reviewed, tested, and reused.
- It brings software engineering practices to infrastructure.

3 PROBLEMS WITH MANUAL INFRASTRUCTURE



TIME
CONSUMING



PRONE TO
HUMAN ERROR



HARD TO
REPRODUCE



NOT EASY TO
COLLABORATE



DIFFICULT TO
SCALE

4 DECLARATIVE INFRASTRUCTURE

- You declare the desired end state.
- Terraform compares the desired state with the real state.
- It creates, updates, or deletes resources to match the desired state.
- You tell Terraform **WHAT** you want, not **HOW** to do it.

TERRAFORM vs CLICKING IN CLOUD CONSOLES

TERRAFORM (CODE)

- ✓ Version controlled
- ✓ Repeatable
- ✓ Reviewable
- ✓ Automatable
- ✓ Consistent across environments
- ✓ Safe and trackable

VS

CLOUD CONSOLE (MANUAL)

- ✗ Not version controlled
- ✗ Hard to reproduce
- ✗ No review process
- ✗ Manual and slow
- ✗ Inconsistent
- ✗ Hard to scale and audit

5 MENTAL MODEL: CODE → PLAN → APPLY → INFRASTRUCTURE



6 JUNIOR ENGINEER TIP



Terraform automates infrastructure, but you still need cloud knowledge. Understand the services, networking, security, and costs behind the code.

REMEMBER:

- ✓ Code is powerful, but understanding is critical.
- ✓ Read the plan before you apply.
- ✓ Small changes can have big impact.
- ✓ Test in safe environments first.
- ✓ Infrastructure is part of your application.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

HOW TERRAFORM ACTUALLY WORKS



1. TERRAFORM CLI

The command-line tool you use to run Terraform commands like init, plan, apply, and destroy.



2. CONFIGURATION FILES

HCL files (.tf) where you declare the infrastructure you want: resources, variables, outputs, etc.



3. PROVIDERS

Plugins that understand the APIs of a specific platform (AWS, Azure, GCP, Kubernetes, etc.).



4. PROVIDER APIS

Terraform talks to the provider, and the provider uses platform APIs to perform actions.



5. TERRAFORM STATE

A state file that tracks the real resources Terraform manages and their current attributes.



6. REAL INFRASTRUCTURE

The actual cloud or on-prem infrastructure created and managed by Terraform.



DESIRED STATE vs ACTUAL STATE

DESIRED STATE

What you declare in your configuration files.

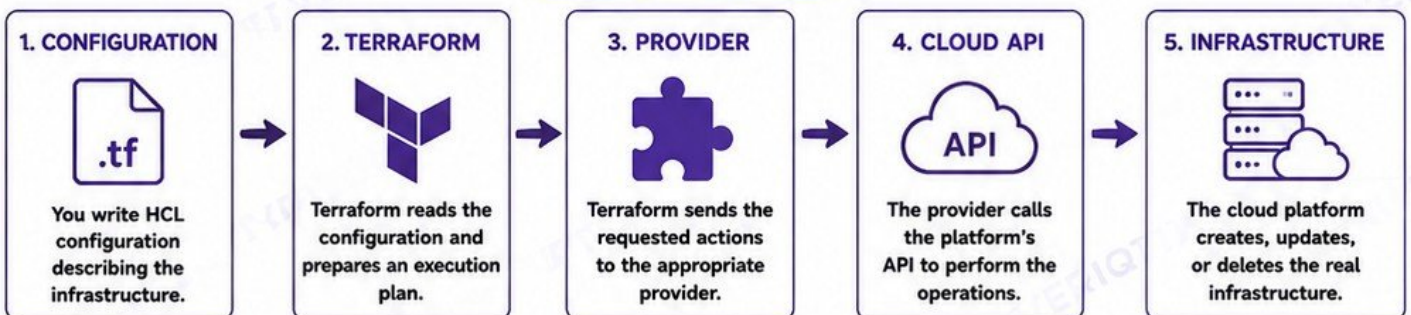


ACTUAL STATE

What exists in the real infrastructure.

Terraform continuously compares desired state with actual state and makes the necessary changes to reach the desired state.

CORE WORKFLOW



THE BIG PICTURE

Terraform is the bridge between your code and your infrastructure. It understands your desired state, works with providers, uses APIs, and keeps your real infrastructure in sync.



JUNIOR ENGINEER TIP

Terraform automates infrastructure, but it does NOT remove the need for cloud knowledge. You must still understand the resources, networking, security, and the services you deploy.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

INSTALLING TERRAFORM AND YOUR FIRST COMMANDS



1 INSTALL TERRAFORM

Download and install Terraform for your operating system.

EXAMPLE (MACOS WITH HOMEBREW)

```
...
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

VERIFY INSTALLATION
terraform version



2 YOUR FIRST COMMANDS

- **terraform version**
Shows the installed Terraform version.
- **terraform -help**
Displays help for all Terraform commands.
- **terraform <command> -help**
Get help for a specific command.



3 CREATE A PROJECT DIRECTORY

```
...
mkdir my-terraform-project
cd my-terraform-project
ls -la
```

Keep your Terraform projects organized in separate directories.



4 .TF FILES

Terraform configuration is written in files ending with .tf

EXAMPLE:

```
...
main.tf variables.tf outputs.tf
providers.tf
```



5 TERRAFORM INIT

```
...
terraform init
```

Initializes your working directory.

- Downloads providers.
- Sets up the backend (if configured).
- Prepares everything for the workflow.



6 TERRAFORM FMT

```
...
terraform fmt
```

Rewrites your configuration files to a canonical format.

- Keeps your code clean and consistent.



7 TERRAFORM VALIDATE

```
...
terraform validate
```

Validates the configuration files in your directory.

- Checks for syntax errors and internal consistency.

8 FIRST HANDS-ON WORKFLOW



1. WRITE
Write your configuration (.tf files).



2. INIT
terraform init
Prepare your working directory.



3. FMT
terraform fmt
Format your code.



4. VALIDATE
terraform validate
Validate your configuration.



5. PLAN
terraform plan
Preview the changes.



6. APPLY
terraform apply
Apply changes to infrastructure.



7. VERIFY
Verify the real infrastructure is created.



JUNIOR ENGINEER TIP

Terraform automates infrastructure, but it does NOT remove the need for cloud knowledge. You must understand the resources, networking, security, and services you are working with.

UNDERSTANDING TERRAFORM CONFIGURATION

TERRAFORM CONFIGURATION IS WRITTEN IN HCL (HASHICORP CONFIGURATION LANGUAGE)

It describes the infrastructure you want, using blocks.

Terraform reads the configuration, compares it with real infrastructure, and makes the necessary changes.

HCL HASHICORP CONFIGURATION LANGUAGE



1. TERRAFORM BLOCKS

Define Terraform settings like required providers and versions.



2. PROVIDER BLOCKS

Configure the provider that gives Terraform access to the platform (AWS, Azure, GCP, etc).



3. RESOURCE BLOCKS

Define the infrastructure resources that Terraform will create or manage.



4. DATA BLOCKS

Read information from existing infrastructure that is not managed by Terraform.



5. VARIABLES

Make your configuration dynamic and reusable. Values can be passed from files or CLI.



6. OUTPUTS

Expose information about your infrastructure after it is created.



7. COMMENTS

Use # to add comments and explain your code.



8. HCL (HASHICORP CONFIGURATION LANGUAGE)

The language Terraform uses.

It is readable, declarative, and consistent.

9. ANATOMY OF A SIMPLE .TF FILE

```
# Configure the AWS provider
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
  profile = "default"
}

resource "aws_s3_bucket" "example" {
  bucket = "my-terraform-bucket-12345"

  tags = {
    Environment = "Dev"
    ManagedBy   = "Terraform"
  }
}

data "aws_ami" "ubuntu" {
  owners      = ["099720109477"]
  most_recent = true
  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-22.04-lts*"]
  }
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

output "bucket_name" {
  description = "Name of the S3 bucket"
  value       = aws_s3_bucket.example.bucket
}
```

TERRAFORM BLOCK (Settings & requirements)

PROVIDER BLOCK (Provider configuration)

RESOURCE BLOCK (Infrastructure resource)

DATA BLOCK (Read existing data)

VARIABLE BLOCK (Input variable)

OUTPUT BLOCK (Return information)



KEY TAKEAWAY

Terraform configuration is made of blocks.

Each block has a specific purpose. Together, they describe your entire infrastructure.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PROVIDERS AND RESOURCES

THE BUILDING BLOCKS OF YOUR INFRASTRUCTURE

1 WHAT A PROVIDER DOES



A provider is a plugin that allows Terraform to interact with an API of a specific platform (cloud, service, or tool).

Providers translate Terraform commands into API calls.

2 POPULAR PROVIDERS



AWS
(amazon)



AZURE
(microsoft)



GOOGLE CLOUD
(google)



KUBERNETES
(hashicorp)

...

AND MANY MORE

3 PROVIDER CONFIGURATION

Configure the provider with the credentials and settings to access your platform.

```
provider "aws" {
  region = "us-east-1"
  profile = "default"
  # or use environment variables / IAM roles
}
```

4 RESOURCE TYPES

Resources are the infrastructure objects you create or manage.



Examples (AWS):

- aws_instance
- aws_s3_bucket
- aws_vpc
- aws_security_group

5 RESOURCE NAMES

The name you give a resource in your configuration.

Example:

```
resource "aws_instance" "web" { ... }
```

↑
TYPE

↑
NAME

6 RESOURCE ADDRESSES

Every resource has a unique address in Terraform.

Format:

<PROVIDER>.<TYPE>.<NAME>

Example:

aws_instance.web

This address is used in plans, state, and outputs.

7 PROVIDER VERSIONS

Always pin provider versions for consistency and stability.



Example:

version = "~> 5.0"

This allows patch updates within major version 5.

8 REQUIRED_PROVIDERS

Define required providers and versions in the terraform block.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

9 EXAMPLE: CREATING AN AWS EC2 INSTANCE

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0" # Amazon Linux 2 AMI (example)
  instance_type = "t3.micro"

  tags = {
    Name        = "web-server"
    Environment = "Dev"
  }
}
```



This configuration creates an EC2 instance.



terraform + required_providers
Declares the AWS provider and pins the version.



provider "aws"
Configures the AWS provider (region in this example).



resource "aws_instance" "web"
Defines an EC2 instance named "web".



Arguments inside the resource
Specify the AMI, instance type, and tags for the instance.



JUNIOR ENGINEER TIP

Understand providers and resources deeply, Terraform only manages what the provider exposes. Strong cloud knowledge helps you design, secure, and operate infrastructure the right way.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



THE TERRAFORM CORE WORKFLOW



Terraform follows a clear and repeatable workflow from writing code to managing real infrastructure.

- 1

WRITE
 Define your infrastructure in code (.tf files).

```
> # Write configuration main.tf
```

You declare the desired state of your infrastructure using HCL.
- 2

TERRAFORM INIT
 Initialize your working directory and download providers.

```
> terraform init
```

Sets up the backend (if any), downloads providers, and prepares everything.
- 3

TERRAFORM FMT
 Format your configuration files to a standard style.

```
> terraform fmt
```

Rewrites files so your code is clean, consistent, and easy to read.
- 4

TERRAFORM VALIDATE
 Validate your configuration files.

```
> terraform validate
```

Checks for syntax errors and internal consistency problems.
- 5

TERRAFORM PLAN
 Preview the changes Terraform will make.

```
> terraform plan
```

Compares your desired state with the current state and shows an execution plan.
- 6

REVIEW
 Carefully review the plan before applying.

```
+ aws_instance.web (create)
~ aws_s3_bucket.logs (update)
- aws_db_instance.old (destroy)
```

Check what will be created, changed, or destroyed. Look for anything unexpected.
- 7

TERRAFORM APPLY
 Apply the changes to reach the desired state.

```
> terraform apply
```

Executes the plan and provisions or updates infrastructure.
- 8

VERIFY
 Verify that the infrastructure is created or updated as expected.

Use cloud consoles, CLI, monitoring tools, or tests to verify everything works.
- 9

DESTROY
 Remove all resources managed by Terraform.

```
> terraform destroy
```

Destroys all resources defined in your configuration.
- 10

MODIFY
 Update your configuration and repeat the workflow.

```
> (edit code and repeat)
```

Infrastructure is versioned and evolves safely.

THE WORKFLOW AT A GLANCE



WHY PRODUCTION ENGINEERS NEVER BLINDLY APPLY CHANGES

- ❌ Plans can include destroying production resources.
- ❌ Small mistakes can cause big outages and data loss.
- ❌ Costs can spike unexpectedly.
- ❌ Security and compliance rules can be violated.
- ❌ Always review, understand, and approve changes.



PRODUCTION ENGINEERING RULE

Never run terraform apply without reviewing the plan first.
Understand the impact. Protect your systems. Own your changes.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

VARIABLES, OUTPUTS, AND LOCAL VALUES



INPUT VARIABLES

Variables make your configuration reusable and dynamic. You define inputs and pass values from different sources.

```
variables.tf

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "environment" {
  description = "Environment name"
  type        = string
}
```



VARIABLE BLOCKS

Define the type, description, default value, and validation.

- ✓ **description** – explains the variable
- ✓ **type** – string, number, bool, list, map, etc.
- ✓ **default** – optional default value
- ✓ **sensitive** – hides value in output
- ✓ **validation** – custom rules

EXAMPLE WITH VALIDATION

```
variable "port" {
  description = "Application port"
  type        = number
  default     = 8080
  validation {
    condition = var.port > 0 && var.port < 65536
    error_message = "Port must be between 1 and 65535."
  }
}
```



TYPES

Common variable types:

- ✓ string
- ✓ number
- ✓ bool
- ✓ list(string)
- ✓ list(number)
- ✓ map(string)
- ✓ object({...})

DEFAULT VALUES

Defaults are used when no value is provided.

```
variables.tf

variable "region" {
  type        = string
  default     = "us-east-1"
}
```



VALUE SOURCES

- terraform.tfvars file
Automatically loaded.
- *.auto.tfvars files
Automatically loaded.
- CLI -var / -var-file
terraform apply -var="key=value"
terraform apply -var-file="prod.tfvars"
- Environment variables
TF_VAR_<variable_name>=value
- Default values in variable block
Used if nothing else is provided.



LOCALS

Locals store expressions or values that can be reused in your configuration.

EXAMPLE:

```
locals.tf

locals {
  name_prefix = "myapp"
  common_tags = {
    Project      = "MyApp"
    ManagedBy    = "Terraform"
    Environment   = var.environment
  }
}
```



OUTPUTS

Outputs expose important information about your infrastructure.

EXAMPLE:

```
outputs.tf

output "instance_id" {
  description = "The ID of the EC2 instance"
  value       = aws_instance.web.id
}

output "public_ip" {
  value       = aws_instance.web.public_ip
  sensitive   = true
}
```



Use sensitive = true for values you don't want displayed.

7 CONFIGURATION PRECEDENCE (HIGH TO LOW)

- 1 CLI -var and -var-file (last one wins)
- 2 Environment variables (TF_VAR_...)
- 3 terraform.tfvars (in root module)
- 4 terraform.tfvars in environment subdirectories
- 5 *.auto.tfvars (alphabetical order)
- 6 Default values in variable blocks



Higher precedence sources override lower precedence sources.

8 AVOID HARDCODED INFRASTRUCTURE VALUES

- ✓ Never hardcode things like region, instance type, VPC IDs, usernames, passwords, etc.
- ✓ Use variables for anything that can change.
- ✓ Makes your code reusable, secure, and environment-agnostic.
- ✓ Hardcoded values lead to duplicated code and human error.



JUNIOR ENGINEER TIP

Use variables for flexibility, locals for reusability, and outputs for visibility. Great Terraform code is dynamic, clean, and environment-friendly.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TERRAFORM STATE EXPLAINED

1 WHAT IS TERRAFORM STATE?

terraform.tfstate is a JSON file that stores the current state of your infrastructure.

- ✓ Tracks every resource Terraform manages.
- ✓ Maps resources in your configuration to real-world infrastructure.
- ✓ Stores resource attributes and metadata.



terraform.tfstate

2 WHY TERRAFORM NEEDS STATE

- ✓ Terraform must know what exists before it can create, update, or delete anything.
- ✓ State allows Terraform to:
 - Plan changes accurately
 - Track resource dependencies
 - Avoid recreating existing resources
 - Detect drift



3 RESOURCE IDs AND MAPPINGS

State stores the unique IDs assigned by the provider and maps them to resources in your configuration.

RESOURCE IN CONFIG	RESOURCE ID IN STATE	PROVIDER
aws_instance.web	i-0a1b2c3d4e5f67890	AWS
aws_s3_bucket.logs	arn:aws:s3:::my-logs-bucket	AWS
azurerm_resource_group.rg	/subscriptions/xxx/rg/my-rg	Azure

4 STATE vs CONFIGURATION

CONFIGURATION (CODE)



- ✓ What you want.
- ✓ Declarative.
- ✓ Version controlled.
- ✓ Human readable.

STATE



VS

- ✓ What exists.
- ✓ Generated by Terraform.
- ✓ Not meant to be edited.
- ✓ Machine readable (JSON).

5 STATE vs ACTUAL INFRASTRUCTURE

STATE



What Terraform believes exists.

VS

ACTUAL INFRASTRUCTURE



What really exists in your cloud environment.

When they differ, it is called **DRIFT**.
Terraform **plan** will detect and show the difference.

6 WHY MANUALLY EDITING STATE IS DANGEROUS



- ✓ You can corrupt the state file.
- ✓ Terraform may lose track of resources.
- ✓ Resources may be recreated or destroyed unexpectedly.
- ✓ Dependencies may break.
- ✓ Recovery can be complex or impossible.

7 SENSITIVE INFORMATION IN STATE



State files can contain:

- ✓ Passwords and tokens
- ✓ Access keys
- ✓ Private IP addresses
- ✓ Endpoint details
- ✓ Certificates and more



Treat state files as **HIGHLY SENSITIVE**.
Secure storage, access control, and encryption are mandatory.

8 PRODUCTION AWARENESS



- ✓ State is critical infrastructure data.
- ✓ Losing or corrupting state can cause outages.
- ✓ Always store state in a secure remote backend.
- ✓ Use encryption, access control, and locking.
- ✓ Back up your state regularly.
- ✓ Protect state like you protect production secrets.



JUNIOR ENGINEER TIP

Terraform state is the source of truth for Terraform.
Understand it, protect it, and never treat it like an ordinary file.

REMOTE STATE AND STATE LOCKING

1 PROBLEMS WITH LOCAL STATE IN TEAMS



- ✗ State file lives on one engineer's machine.
- ✗ State cannot be shared safely.
- ✗ Risk of accidental overwrites.
- ✗ No locking: multiple people can run apply at the same time.
- ✗ Hard to recover if the state file is lost or deleted.
- ✗ Not suitable for CI/CD pipelines.

2 REMOTE BACKENDS

Remote backends store your state file in a remote system designed for sharing, security, and reliability.



POPULAR BACKENDS

- AWS S3 + DynamoDB
- Azure Blob Storage + Azure Table Storage
- Google Cloud Storage
- Terraform Cloud / Enterprise
- Self-hosted solutions

3 SHARED STATE

Remote state allows your team (or pipeline) to always work with the same, latest state.



- ✓ Single source of truth.
- ✓ Consistent plans and applies.
- ✓ Safe for teams and automation.

4 STATE LOCKING



State locking prevents multiple users from running operations that could corrupt the state.

- ✓ Only one operation can modify state at a time.
- ✓ Lock is acquired before apply.
- ✓ Lock is released when operation completes.



Without locking, concurrent applies can overwrite state and destroy resources.

5 ENCRYPTION



Remote backends encrypt state files both in transit and at rest.

- ✓ Protects sensitive data (IDs, endpoints, secrets).
- ✓ Encryption is mandatory for production.

6 ACCESS CONTROL



Restrict who can read or modify the state.

- ✓ Use IAM roles / policies.
- ✓ Least privilege access.
- ✓ Separate access for read vs write.

7 VERSIONING AND RECOVERY



Remote backends keep version history and enable recovery.

- ✓ Revert to previous state if something goes wrong.
- ✗ Protects against accidental deletes or corruption.
- ✓ Audit and trace changes.

8 BACKEND CONFIGURATION (EXAMPLE: AWS S3 + DYNAMODB)

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state-bucket"
    key    = "envs/prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-lock-table"
    encrypt    = true
  }
}
```

KEY SETTING EXPLAINED

- bucket → S3 bucket to store state
- key → Path to state file
- region → AWS region
- dynamodb_table → Table used for locking
- encrypt → Enable encryption at rest



Always use a remote backend for team and production environments.

9 WHY STATE FILES SHOULD NOT BE COMMITTED TO GIT



- ✗ State contains sensitive information.
- ✗ Hard to manage merge conflicts.
- ✗ Risk of leaking secrets.
- ✗ Large state files bloat your repo.
- ✗ Backups and versioning are better handled by remote backend.



Add this to your .gitignore

*.tfstate

10 TEAM WORKFLOW MENTAL MODEL



REMOTE STATE + LOCKING = SAFETY, CONSISTENCY, AND TEAM SCALE
It ensures your infrastructure is managed as a true team asset, not a local file.



JUNIOR ENGINEER TIP

Always use remote state with locking, encryption, and access control. State is critical infrastructure data. Protect it like production secrets.



TERRAFORM DEPENDENCIES AND RESOURCE GRAPH

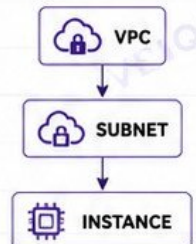
1 IMPLICIT DEPENDENCIES

- Terraform automatically understands dependencies when one resource references another.
- If Resource B uses an attribute from Resource A, Terraform knows A must be created first.

```
resource "aws_instance" "app" {
  subnet_id = aws_subnet.public.id
}
```

2 RESOURCE REFERENCES

- Using references creates a natural dependency.
- Terraform builds the graph based on these references.



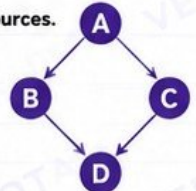
3 EXPLICIT DEPENDS_ON

- Use depends_on when Terraform cannot detect the dependency automatically.
- Use only when absolutely necessary. Overuse leads to slower and fragile plans.

```
resource "aws_instance" "app" {
  ami = "ami-12345"
  depends_on = [
    aws_iam_role.app_role
  ]
}
```

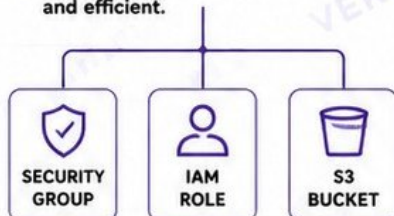
4 TERRAFORM DEPENDENCY GRAPH

- Terraform builds a graph of all resources.
- Nodes = resources
- Edges = dependencies
- Terraform walks the graph to determine the correct creation order.



5 PARALLEL RESOURCE CREATION

- Resources without dependencies are created in parallel.
- This makes Terraform fast and efficient.



These can be created at the same time.

6 DEPENDENCY ORDERING

- Terraform creates resources in the correct order based on the graph.
- Destroy order is the reverse of creation order.

✓ CORRECT ORDER

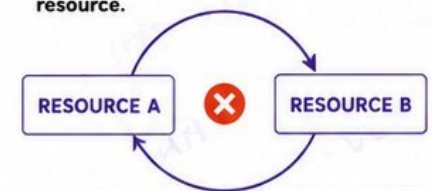


✗ WRONG ORDER (WILL FAIL)



7 CIRCULAR DEPENDENCIES

- A circular dependency occurs when Resource A depends on Resource B, and Resource B depends on A.
- Terraform cannot create either resource.



TERRAFORM ERROR:
CYCLE DETECTED

8 WHY UNNECESSARY DEPENDS_ON CREATES POOR DESIGNS



- ✗ Forces resources to wait unnecessarily.
- ✗ Slows down plans and applies.
- ✗ Creates hidden, hard-to-maintain dependencies.
- ✗ Makes configurations fragile.
- ✗ Leads to unexpected replacements.
- ✗ Trust Terraform's automatic dependency detection whenever possible.

9 EXAMPLE DIAGRAM: VPC → SUBNET → INSTANCE



WHAT HAPPENS?

- ✓ VPC is created first.
- ✓ Subnet is created after VPC.
- ✓ Instance is created after Subnet.
- ✓ On destroy, Terraform removes in reverse order.

EXISTING INFRASTRUCTURE

1 RESOURCE vs DATA SOURCE

RESOURCE (CREATE)	DATA SOURCE (READ)
 - Creates new infrastructure - Managed and destroyed by Terraform - Has a lifecycle - Example: <code>aws_instance</code>, <code>aws_vpc</code>	 - Reads existing infrastructure - Not managed by Terraform - No lifecycle - Example: <code>data "aws_vpc"</code>, <code>data "aws_ami"</code>



Data sources help Terraform learn about resources that already exist outside of Terraform.

2 READING EXISTING INFRASTRUCTURE

- Use data blocks to look up information about existing resources.
- Terraform uses that information to build relationships and pass values.
- Data sources do not create, update, or delete anything.

```
data "aws_vpc" "existing" {
  id = "vpc-0abc123def456ghi7"
}

data "aws_subnet" "public" {
  id = "subnet-0a1b2c3d4e5f6g7h8"
}
```



EXISTING
CLOUD
INFRASTRUCTURE

3 COMMON DATA SOURCE EXAMPLES

LOOKING UP AMIS

```
data "aws_ami" "ubuntu" {
  owners      = ["099720109477"]
  most_recent = true
  filter {
    name   = "name"
    values = ["ubuntu/images/*ubuntu-22.04*"]
  }
}
```



Find the latest Ubuntu 22.04 AMI from AWS.

EXISTING VPCS

```
data "aws_vpc" "main" {
  filter {
    name   = "tag:Name"
    values = ["production-vpc"]
  }
}
```



Read an existing VPC by tag.

EXISTING SUBNETS

```
data "aws_subnets" "public" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.main.id]
  }
  filter {
    name   = "tag:Tier"
    values = ["public"]
  }
}
```



Get all public subnets in a VPC.

4 REFERENCING DATA-SOURCE ATTRIBUTES

```
data "aws_vpc" "main" {
  filter {
    name   = "tag:Name"
    values = ["production-vpc"]
  }
}

resource "aws_instance" "app" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.public.ids[0]
  vpc_security_group_ids = [data.aws_security_group.app.id]
}
```

Attribute: id

Attribute: ids (list)

Attribute: id

5 WHEN TO CREATE vs WHEN TO DISCOVER

CREATE (resource)	DISCOVER (data source)
- You need Terraform to manage the full lifecycle. - The resource is specific to this environment. - You want consistent, repeatable infrastructure. - Terraform should handle updates and deletions.	- The resource already exists outside Terraform. - The resource is shared across multiple systems or teams. - You only need information about the resource. - Terraform should not modify or destroy it.

6 KEY TAKEAWAYS



- Data sources allow Terraform to read existing infrastructure.
- They help you connect new infrastructure to what is already there.
- Use resources to build and manage, data sources to look up and reference.
- Good designs use data sources to avoid duplication and reduce risk.



JUNIOR ENGINEER TIP

Always inspect the data you read. Use outputs and terraform console to confirm values before relying on them in resources.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LOOPS AND CONDITIONAL INFRASTRUCTURE

1 KEY CONCEPTS

- Terraform lets you create multiple similar resources using loops and conditions.
- This reduces repetition, improves maintainability, and enables dynamic infrastructure.



COUNT



FOR_EACH



FOR EXPRESSIONS



CONDITIONAL EXPRESSIONS



MAPS



SETS



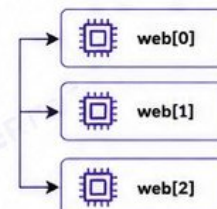
DYNAMIC CONFIGURATION

2 COUNT

- Creates multiple resources based on a number.
- Uses numeric index (0,1,2...).

```
resource "aws_instance" "web" {
  count = 3
  ami   = "ami-123456"
  instance_type = "t3.micro"
  tags = {
    Name = "web-${count.index}"
  }
}
```

COUNT = 3

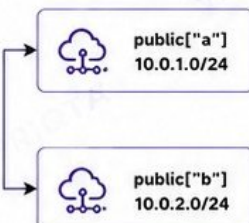


3 FOR_EACH

- Creates multiple resources from a map or set.
- Uses keys (string identifiers) instead of numeric indexes.

```
resource "aws_subnet" "public" {
  for_each = {
    a = "10.0.1.0/24"
    b = "10.0.2.0/24"
  }
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value
  availability_zone = "us-east-1a"
  tags = {
    Name = "public-${each.key}"
  }
}
```

FOR_EACH (MAP)



4 FOR EXPRESSIONS

- Transform, filter, or build collections.
- Useful with maps, lists, and sets.

```
# Convert list to upper case
locals {
  envs = ["dev", "staging", "prod"]
  envs_upper = [for e in local.envs : upper(e)]
}

# Filter even numbers
locals {
  evens = [for n in [1,2,3,4,5] : n if n % 2 == 0]
}
```

OUTPUT

```
envs_upper = [
  "DEV",
  "STAGING",
  "PROD"
]

evens = [2, 4]
```

5 CONDITIONAL EXPRESSIONS

- Create resources or values based on a condition.
- Syntax: condition ? true_val : false_val

```
resource "aws_instance" "example" {
  count = var.create_instance ? 1 : 0
  ami   = "ami-123456"
  instance_type = "t3.micro"
}
```

6 MAPS vs SETS



MAPS

- Key-value pairs.
- Access by key.
- Use with for_each.
- Example: { a = "1", b = "2" }



SETS

- Unique values only.
- No keys or indexes.
- Use with for_each.
- Example: toset(["a", "b", "c"])

7 DYNAMIC CONFIGURATION

- Use dynamic blocks to create nested structures.
- Useful for repeatable nested blocks like ingress rules.

```
resource "aws_security_group" "sg" {
  name = "example"

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port = ingress.value.from_port
      to_port   = ingress.value.to_port
      protocol  = ingress.value.protocol
      cidr_blocks = [ingress.value.cidr]
    }
  }
}
```

8 CREATING MULTIPLE RESOURCES SAFELY

- ✓ Prefer for_each over count when possible (stable identity).
- ✓ Use meaningful keys.
- ✓ Avoid hardcoded indexes.
- ✓ Keep loops simple and readable.
- ✓ Validate inputs to prevent unexpected results.



9 COUNT vs FOR_EACH

FEATURE	COUNT	FOR_EACH
Input Type	Number	Map or Set
Resource Address	resource.name[0]	resource.name["key"]
Identity Stability	Can change if order changes	Stable (key-based)
Best For	Simple, identical resources	Complex, keyed resources
Example Use Case	3 web servers	Subnets by AZ name

10 COMMON IDENTITY AND INDEXING MISTAKES

- ✗ Using count and removing an item (indexes shift and destroy wrong resource).
- ✗ Using lists with for_each (lists have no keys).
- ✗ Creating resources in a different order than expected.
- ✗ Hardcoding indexes (e.g., resource[0]).
- ✗ Not using stable keys (e.g., names, IDs).
- ✗ Changing keys in for_each (forces resource recreation).



**BAD DESIGNS
LEAD TO
DOWNTIME AND
DATA LOSS!**



JUNIOR ENGINEER TIP

Loops and conditions are powerful. Use them wisely to build flexible, predictable, and maintainable infrastructure.



SUMMARY

- Use loops to reduce repetition.
- Use conditions to control creation.
- Use the right tool for stable and safe infrastructure.

SCAN TO PRACTICE
MORE ON TERRAFORM



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TERRAFORM MODULES

BUILD ONCE. REUSE EVERYWHERE.

1 WHAT A MODULE IS



A module is a container for multiple Terraform resources that are used together. Modules help you organize, reuse, and share infrastructure as a blueprint.

2 ROOT MODULE



- The main module in your configuration.
- Calls child modules.
- Defines providers, backend, and other global settings.

3 CHILD MODULES



- Reusable building blocks.
- Contain resources, variables, and outputs.
- Called by the root module or other modules.

4 INPUTS (VARIABLES)



- Modules accept inputs using variables.
- Make modules flexible and reusable.
- Passed from the root module.

5 OUTPUTS



- Modules expose output values.
- Useful information returned to the root module.
- Can be used by other resources or modules.

6 MODULE SOURCES

Tells Terraform where to find the module.

- Local paths
- Terraform Registry (public or private)
- Git repositories
- HTTP(S) sources

7 LOCAL MODULES



- Stored in your repository.
- Fast, simple, and good for internal modules.
- Example source:

```
source = "../modules/vpc"
```

8 REGISTRY MODULES



- Published to Terraform Registry.
- Easy to discover, version, and reuse.
- Example source:

```
source = "hashicorp/vpc/aws"
```

9 VERSION PINNING



- Always pin module versions to avoid unexpected changes.
- Use exact versions for production.
- Example:

```
version = ">= 3.2"
```

10 REUSABLE INFRASTRUCTURE



- Modules let teams build standard, secure, and consistent infrastructure.
- Save time, reduce errors, and enforce best practices.
- Share modules across projects and teams.

11 WHEN A MODULE BECOMES TOO COMPLICATED



- Too many responsibilities.
 - Too many variables and outputs.
 - Hard to understand or maintain.
 - Frequent breaking changes.
 - Hard for new engineers to use.
- Solution: Split it into smaller, focused modules.

HOW MODULES WORK TOGETHER

ROOT MODULE



```
main.tf
variables.tf
outputs.tf
```

PASS INPUTS
(VARIABLES)

CHILD MODULE



```
variables.tf
resources.tf
outputs.tf
```

RETURN OUTPUTS
(VALUES)

RESOURCES IN CLOUD



EXAMPLE: CALLING A MODULE

```
module "vpc" {
  source = "../modules/vpc"
  vpc_cidr = "10.0.0.0/16"
  azs     = ["us-east-1a", "us-east-1b"]
}

# Using output from the module
resource "aws_instance" "app" {
  subnet_id = module.vpc.private_subnet_ids[0]
}
```



JUNIOR ENGINEER TIP

Start simple. Build small, single-purpose modules. Great modules are easy to use, easy to understand, and solve one problem really well.



@veriqta



veriqta



veriqta



veriqta



@veriqta

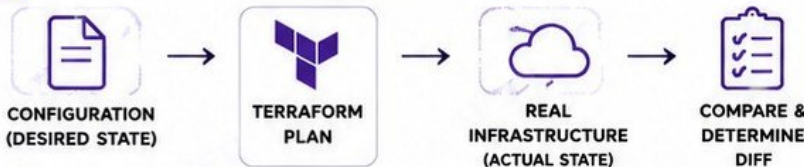


@veriqta

LIFECYCLE AND INFRASTRUCTURE CHANGES

CONTROL CHANGE. REDUCE RISK. SHIP CONFIDENTLY.

1 HOW TERRAFORM DETECTS CHANGES



Terraform compares your configuration with the real infrastructure. If there is a difference, Terraform shows the plan of actions needed to reach the desired state.

2 CHANGE ACTIONS: WHAT TERRAFORM CAN DO



CREATE

Resource does not exist. Terraform creates it.



UPDATE IN PLACE

Resource exists and can be updated without destroying it.



REPLACE

Resource change requires destroying the old one and creating a new one.



DESTROY

Resource is removed from configuration and will be destroyed.

3 LIFECYCLE META-ARGUMENTS

```

resource "aws_instance" "web" {
  ami          = "ami-123456"
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = true
    ignore_changes        = [
      tags["LastModified"],
      user_data
    ]
  }
}
  
```

create_before_destroy

Create the new resource before destroying the old one to avoid downtime.

prevent_destroy

Prevents Terraform from destroying the resource by accident.

ignore_changes

Ignores changes to specific attributes to avoid unnecessary updates.

4 PRODUCTION RISK OF REPLACEMENT

Some changes require resource replacement. This can cause:

- Downtime
- Data loss
- IP/Endpoint changes
- Service disruption
- Cost impact



EXAMPLES THAT FORCE REPLACEMENT

- ✗ Changing instance_type (sometimes)
- ✗ Changing name
- ✗ Changing subnet
- ✗ Changing some immutable attributes
- ✗ Some provider limitations



ALWAYS READ THE PLAN CAREFULLY

Understand what will be created, updated, replaced, or destroyed before you apply.

3 READING A TERRAFORM PLAN BEFORE APPROVAL

```

$ terraform plan
Terraform will perform the following actions:
# aws_instance.web will be updated in-place
~ resource "aws_instance" "web" {
  ~ instance_type = "t2.micro" -> "t3.micro"
}

# aws_s3_bucket.logs will be replaced
-/+ resource "aws_s3_bucket" "logs" {
  - bucket = "old-bucket-name" -> null
  + bucket = "new-bucket-name"
}

Plan: 1 to add, 1 to change, 1 to destroy.
  
```



Addition
Resource will be created.



In-place Update
Resource will be updated.



Replacement
Destroy old and create new.



Destruction
Resource will be destroyed.

PLAN REVIEW CHECKLIST

- ✓ What resources are being created?
- ✓ What is being updated in-place?
- ✓ What will be replaced? Why?
- ✓ What will be destroyed?
- ✓ Will this cause downtime?
- ✓ Will this increase cost?
- ✓ Are dependencies affected?
- ✓ Does this match the change request?



JUNIOR ENGINEER TIP

Terraform is powerful, but with power comes responsibility.

Always understand the impact of your changes. Read the plan. Ask questions. Protect production.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

IMPORTING AND MANAGING EXISTING INFRASTRUCTURE

ADOPT. MANAGE. PROTECT WHAT ALREADY EXISTS.

1 INFRASTRUCTURE CREATED OUTSIDE TERRAFORM



Many resources are created manually, by other tools, or by other teams. Terraform can manage them by adopting them into its state.

2 TERRAFORM IMPORT CONCEPTS



- Import adds an existing real-world resource into Terraform state.
- It does NOT create or change infrastructure.
- After import, Terraform can track and manage it.
- You must still write matching configuration.

3 IMPORT BLOCKS AND WORKFLOWS (TERRAFORM v1.5+)

IMPORT WORKFLOW

- 1  Identify the existing resource (ID required)
- 2  Write the resource configuration in Terraform
- 3  Import the resource into Terraform state
- 4  Plan and verify no unexpected changes
- 5  Apply and start managing safely

IMPORT BLOCK EXAMPLE

```
import {
  to = aws_s3_bucket.logs
  id = "my-existing-bucket"
}
```

CLI IMPORT EXAMPLE

```
terraform import
aws_s3_bucket.logs
my-existing-bucket
```

NOTE

The resource address (`to`) must match the address in your configuration.

4 MATCHING CONFIGURATION TO EXISTING RESOURCES

Your Terraform configuration must match the real resource's settings.



GOOD MATCH

Configuration matches the real resource

```
resource "aws_s3_bucket" "logs" {
  bucket = "my-existing-bucket"
  acl    = "private"
  force_destroy = false
  tags = {
    Environment = "prod"
  }
}
```



BAD MATCH

Configuration does not match the real resource

```
resource "aws_s3_bucket" "logs" {
  bucket = "my-existing-bucket"
  acl    = "public-read"
  force_destroy = true
}
```



WARNING

If configuration does not match, Terraform will try to **CHANGE** or **REPLACE** the resource on the next apply.

5 INSPECTING STATE AFTER IMPORT



Use these commands to inspect what Terraform now manages.

```
$ terraform state list
$ terraform state show
aws_s3_bucket.logs
$ terraform show
$ terraform plan
```

6 DRIFT AFTER IMPORT

Drift happens when someone changes the resource outside Terraform after import.

EXAMPLES

- Manual changes in cloud console
- Other tools modifying resources
- Different default values

Terraform plan will show the drift.



7 ADOPTION STRATEGY

- ✓ Inventory existing resources
- ✓ Decide what to manage
- ✓ Write and test configuration
- ✓ Import one resource at a time
- ✓ Plan and verify (no surprises)
- ✓ Repeat safely
- ✓ Document and set standards
- ✓ Automate and enforce

8 WHY IMPORTING PRODUCTION INFRASTRUCTURE REQUIRES CARE



- High blast radius
- Configuration mismatch can cause changes or destruction
- State corruption risk
- Locking and teamwork are critical
- Always backup state before importing



JUNIOR ENGINEER TIP

Import is not just a command. It is a process of taking ownership. Be patient, verify everything, and never rush in production.



PRODUCTION REMINDER

- Always take a state backup before importing.
- Import small, validate often, and communicate with your team.
- When in doubt, test in non-production first.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TERRAFORM SECURITY AND SECRETS

SECURE YOUR CODE. PROTECT YOUR INFRASTRUCTURE.

1 NEVER HARDCODE CREDENTIALS



```
provider "aws" {
  access_key = "AKIA..."
  secret_key = "abcd1234"
  region     = "us-east-1"
}
```

- Never put secrets in your code.
- Never commit secrets to Git.
- Never share secrets in chat or tickets.
- Use secure methods to provide credentials.

2 PROVIDER AUTHENTICATION



- Configure providers to authenticate securely to your cloud.
- Use supported authentication mechanisms.
- Avoid long-lived static credentials.
- Rotate credentials regularly.

3 ENVIRONMENT CREDENTIALS



- Use environment variables.
- Example: AWS_ACCESS_KEY_ID
- Use credential files outside your Terraform code.
- Different credentials per environment (dev, staging, prod).

4 WORKLOAD IDENTITIES AND ROLES



- Use IAM roles, OIDC, or Workload Identity where possible.
- Short-lived credentials reduce risk.
- Terraform should assume roles, not use static keys.
- Follow least privilege for identities.

5 SECRET MANAGERS



- Store secrets in a dedicated secret manager.
- Examples: AWS Secrets Manager, HashiCorp Vault, Azure Key Vault, GCP Secret Manager.
- Pull secrets at runtime or during apply.
- Never store secrets in tfvars files.
- Limit access to secrets.

6 SENSITIVE VARIABLES



- Mark variables as sensitive to hide them in CLI output.

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

This hides values in the plan and output, but does NOT encrypt them in state.

7 STATE SECURITY



- Terraform state contains ALL resource data.
- It may include secrets, endpoints, and IDs.
- Store state in secure remote backends.
- Encrypt state at rest.
- Restrict access with IAM policies.
- Enable versioning and backups.

8 LEAST PRIVILEGE



- Give Terraform only the permissions it needs.
- Avoid admin or broad permissions.
- Limit actions to specific resources and regions.
- Review and audit permissions regularly.

9 PROVIDER AND MODULE TRUST



- Use official providers from trusted sources.
- Pin provider versions.
- Review module source and code before use.
- Avoid unverified modules from unknown sources.

10 .GITIGNORE - DO NOT COMMIT SENSITIVE FILES



```
# Terraform
.terraform/
*.tfstate
**tfstate.*
crash.log
override.tf
terraform.tfvars
*.auto.tfvars
.terraform.lock.hcl
```

- Always ignore state files.
- Ignore tfvars files.
- Ignore credentials and override files.
- Keep your repo clean and safe.

11 SECURITY SCANNING



- Scan your Terraform code for security issues.
- Tools: tfsec, Checkov, Terrascan, Snyk IaC.
- Integrate scans into CI/CD pipelines.
- Fix issues before apply.

12 SENSITIVE = TRUE DOES NOT ENCRYPT STATE



- sensitive = true hides values in CLI output.
- Values are still stored in plain text in the state file.
- Anyone with state access can read the secret.
- Secure the state, not just the code!



**SECURITY IS NOT OPTIONAL IN INFRASTRUCTURE AS CODE.
A LEAKED SECRET CAN DESTROY SYSTEMS AND TRUST.
PROTECT YOUR CODE, YOUR STATE, AND YOUR CREDENTIALS.**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TERRAFORM IN CI/CD

AUTOMATE INFRASTRUCTURE. CONTROL CHANGES.

1 CI/CD PIPELINE OVERVIEW



2 AUTOMATED CHECKS (RUN BY CI)

	terraform fmt -check	Ensures consistent code formatting.
	terraform validate	Validates Terraform syntax and configuration.
	SECURITY CHECKS	Scan for vulnerabilities, misconfigurations, and secrets.
	COST / POLICY CHECKS	Validate cost, tags, and policy compliance.
	CUSTOM CHECKS	tflint, tfsec, checkov, OPA, Sentinel, etc.

3 PLAN, REVIEW, APPROVE, APPLY



terraform plan

Generates an execution plan (what will be created, updated, or destroyed).



HUMAN REVIEW

Engineers review the plan for accuracy, safety, cost, and compliance.



APPROVAL

Only authorized approvers can approve the run.



terraform apply

Applies the changes exactly as reviewed and approved.

PLAN OUTPUT EXAMPLE

```
$ terraform plan
...
Plan: 2 to add, 1 to change,
0 to destroy.

+ aws_instance.web
+ aws_s3_bucket.logs
~ aws_security_group.web

(Plan output truncated)
```



KEY POINT

The plan is the contract. Apply should only run what was reviewed.

4 CI/CD CREDENTIALS

	Use least-privilege service accounts or IAM roles.
	Store credentials securely (Vault, AWS OIDC, Azure Workload Identity, GitHub OIDC, etc).
	Never hardcode credentials in code or pipelines.
	Rotate credentials regularly.

5 SEPARATING PLAN FROM APPLY



This ensures the applied changes are exactly what were reviewed.

6 WHY PRODUCTION APPLIES NEED CONTROLS



PREVENT ACCIDENTS

Bad code can delete critical resources.



CONTROL COSTS

Unreviewed changes can create unexpected charges.



ENSURE COMPLIANCE

Meet security, tagging, and governance requirements.



ENSURE ACCOUNTABILITY

Every change is traceable to a person, review, and approval.



ENABLE SAFE RECOVERY

Reviewed plans and history help with quick rollbacks.



JUNIOR ENGINEER TIP

Automate everything you can, but never skip review and approval for production. Terraform gives you power. Process and controls keep production safe.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TERRAFORM TROUBLESHOOTING AND DRIFT

FIND THE PROBLEM. UNDERSTAND THE CAUSE. FIX IT RIGHT.

1 COMMON PROBLEMS

 FAILED TERRAFORM INIT Backend errors, network issues, or missing provider plugins.	 PROVIDER ERRORS Unsupported arguments, deprecated fields, or provider bugs.	 AUTHENTICATION FAILURES Invalid credentials, expired tokens, or insufficient permissions.	 INVALID REFERENCES Wrong resource names, typos, or references to resources that don't exist.	 DEPENDENCY FAILURES Resources created in the wrong order or missing dependencies.
 STATE LOCK PROBLEMS State is locked by another process or a crashed operation.	 DRIFT Actual infrastructure differs from the state (manual changes).	 UNEXPECTED RESOURCE REPLACEMENT Changes that force recreation of critical resources.	 HARD TO DEBUG PLANS Large plans are difficult to read and understand the impact.	 UNCLEAR ERROR MESSAGES Errors can be misleading. You must read them carefully.

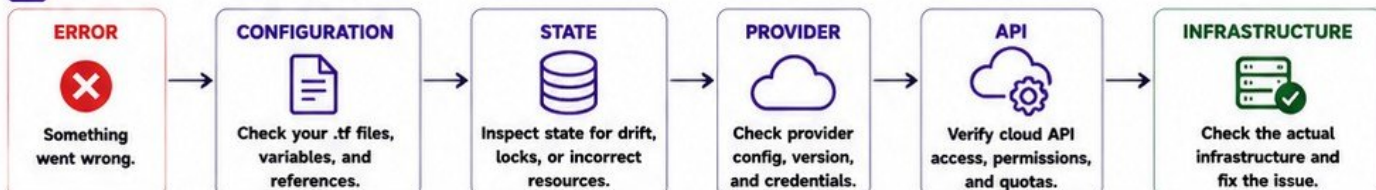
2 ESSENTIAL COMMANDS FOR TROUBLESHOOTING

terraform plan	terraform state list	terraform state show <address>	terraform refresh	terraform graph	terraform taint <address>	terraform import <address> <id>	terraform force-unlock <lock-id>
 Shows proposed changes. First tool to understand issues.	 Lists all resources tracked in the state.	 Shows details of a specific resource in the state.	 Syncs state with real infrastructure.	 Visualizes resource dependencies.	 Forces recreation of a resource.	 Bring existing infrastructure under Terraform.	 Manually unlock state (use with caution!).

3 READING ERROR MESSAGES

- 
 - ✓ **READ CAREFULLY:**
The first error is usually the root cause.
 - ✓ **IGNORE NOISE:**
Some errors are side effects. Focus on the real issue.
- ✓ **CHECK CONTEXT:**
Look at the resource, line number, and module path.
 - ✓ **USE THE DOCUMENTATION:**
Error messages often suggest the next step.
- ✓ **RUN WITH DEBUG:**
terraform plan -no-color -debug for deep troubleshooting.
 - ✓ **SEARCH SMART:**
Copy the exact error message (or key part) into search.

4 TROUBLESHOOTING FLOW



JUNIOR ENGINEER TIP

Always isolate the problem step by step. Change one thing at a time. Understand the root cause, not just the fix.



@veriqta



veriqta



veriqta



veriqta



@veriqta



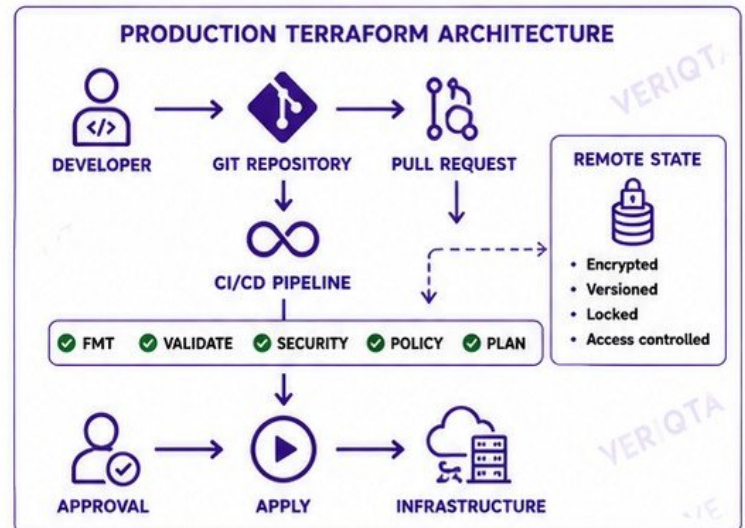
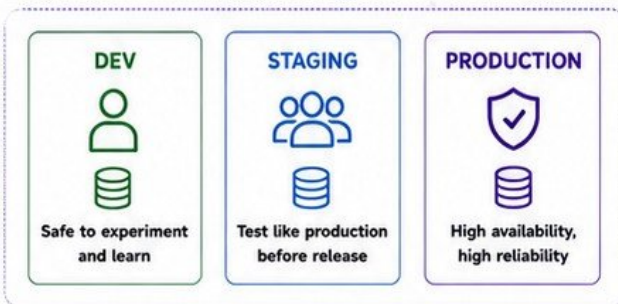
@veriqta

PRODUCTION TERRAFORM ARCHITECTURE

DESIGN FOR SCALE. BUILD FOR SAFETY. OPERATE WITH CONFIDENCE.

1 ENVIRONMENTS

Keep environments separate to reduce risk and blast radius. Use consistent patterns across Dev, Staging, and Production.



2 ENVIRONMENT ISOLATION

- Separate cloud accounts or subscriptions.
- Separate networks.
- Separate secrets.
- Separate state.

3 STATE ISOLATION

- One state per environment.
- Separate state for different components.
- Use remote backend with locking and encryption.

4 REPOSITORY STRUCTURE

- Environment folders (dev/, staging/, prod/).
- Modules in /modules or separate repo.
- Clear, consistent layout.

5 REUSABLE MODULES

- Build modules for repeatable patterns.
- Keep modules small, focused and composable.
- Share modules across teams and projects.

6 VERSION PINNING

- Pin provider versions.
- Pin module versions.
- Avoid unexpected changes.

```
required_providers {
  aws = { source = "hashicorp/aws"
    version = "~> 5.0" }
}
```

7 REMOTE STATE

- Store state in S3, GCS, Azure Storage, etc.
- Enable encryption.
- Enable versioning.
- Enable state locking.

8 CI/CD

- Automate fmt, validate, security, and plan.
- Store plan as artifact.
- Separate plan and apply stages.

9 CODE REVIEW

- Review all changes.
- Check for risk and best practices.
- Never bypass review for production.

10 POLICY & SECURITY CHECKS

- Enforce policies with Sentinel, OPA, or Checkov.
- Scan for misconfigurations.
- Block unsafe changes.

11 CHANGE APPROVAL

- Require human approval for applies.
- Use protected branches and required reviewers.
- Maintain an audit trail.

12 RECOVERY PLANNING

- Backup state regularly.
- Document recovery steps.
- Test restore process.
- Have rollback and break-glass procedures.

13 AVOID ONE ENORMOUS STATE FILE

- Split by domain or component.
- Smaller states are faster and safer.
- Easier to manage and troubleshoot.

14 PRODUCTION TERRAFORM PRINCIPLES



ISOLATE
Environments, state, and access.



AUTOMATE
Build pipelines and checks.



REVIEW
Every change matters.



SECURE
Least privilege, encrypt everything.



REUSE
Modules make you faster and consistent.



PLAN & RECOVER
Expect failure. Prepare for it.



JUNIOR ENGINEER TIP

Production Terraform is about discipline, not just code. Great architecture today prevents incidents tomorrow.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



FINAL PRODUCTION LAB, BUILD AND RECOVER INFRASTRUCTURE

BUILD IT RIGHT. BREAK IT ON PURPOSE. FIX IT WITH CONFIDENCE.

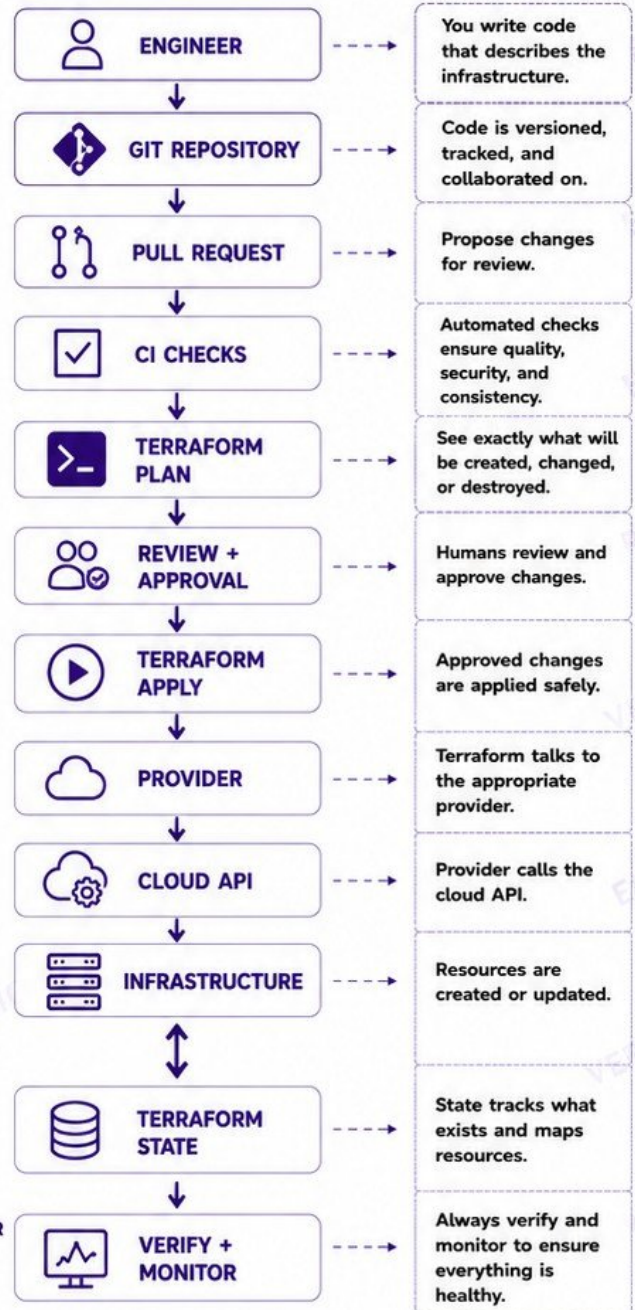
1 PRODUCTION LAB STEPS

- 1 Build a reusable Terraform project
- 2 Configure provider versions
- 3 Create networking (VPC, Subnets, etc.)
- 4 Create compute resources (EC2 / VM / Containers)
- 5 Use variables and locals
- 6 Add outputs
- 7 Use a reusable module
- 8 Configure remote state
- 9 Format and validate
- 10 Generate a plan
- 11 Review proposed changes
- 12 Apply infrastructure
- 13 Verify the deployed resources
- 14 Introduce configuration drift (manual change)
- 15 Detect the drift (terraform plan)
- 16 Diagnose an unexpected replacement
- 17 Restore the desired configuration
- 18 Verify state and infrastructure

3 KEY PRINCIPLES

AUTOMATE EVERYTHING Use code and CI/CD to eliminate manual changes.	ISOLATE EVERYTHING Separate environments and state for safety and scale.	MAKE IT REUSABLE Use modules, variables, and standards.
SECURE EVERYTHING Least privilege, secrets management, and policy enforcement.	REVIEW EVERYTHING Plans, code, and changes must always be reviewed.	PLAN & RECOVER EVERYTHING Backups, state versioning, and rollback strategies.

2 FINAL PRODUCTION MENTAL MODEL



JUNIOR ENGINEER TIP

Practice building, breaking, and recovering. That is how you build real confidence as a Terraform engineer.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta